# LangGraph Marketing Pipeline

Flow, architecture and model choices — drawn so you can redraw them

Built by Siva Borra as a guided learning exercise. Phases 1–4, 30 October – 5 November 2026. A learning exercise that closes a real deployment gap — not a commercial product, not a production system, nothing published anywhere real. Every number here is measured from the runs in the learning journal; nothing is invented.

## How to use this document in an interview

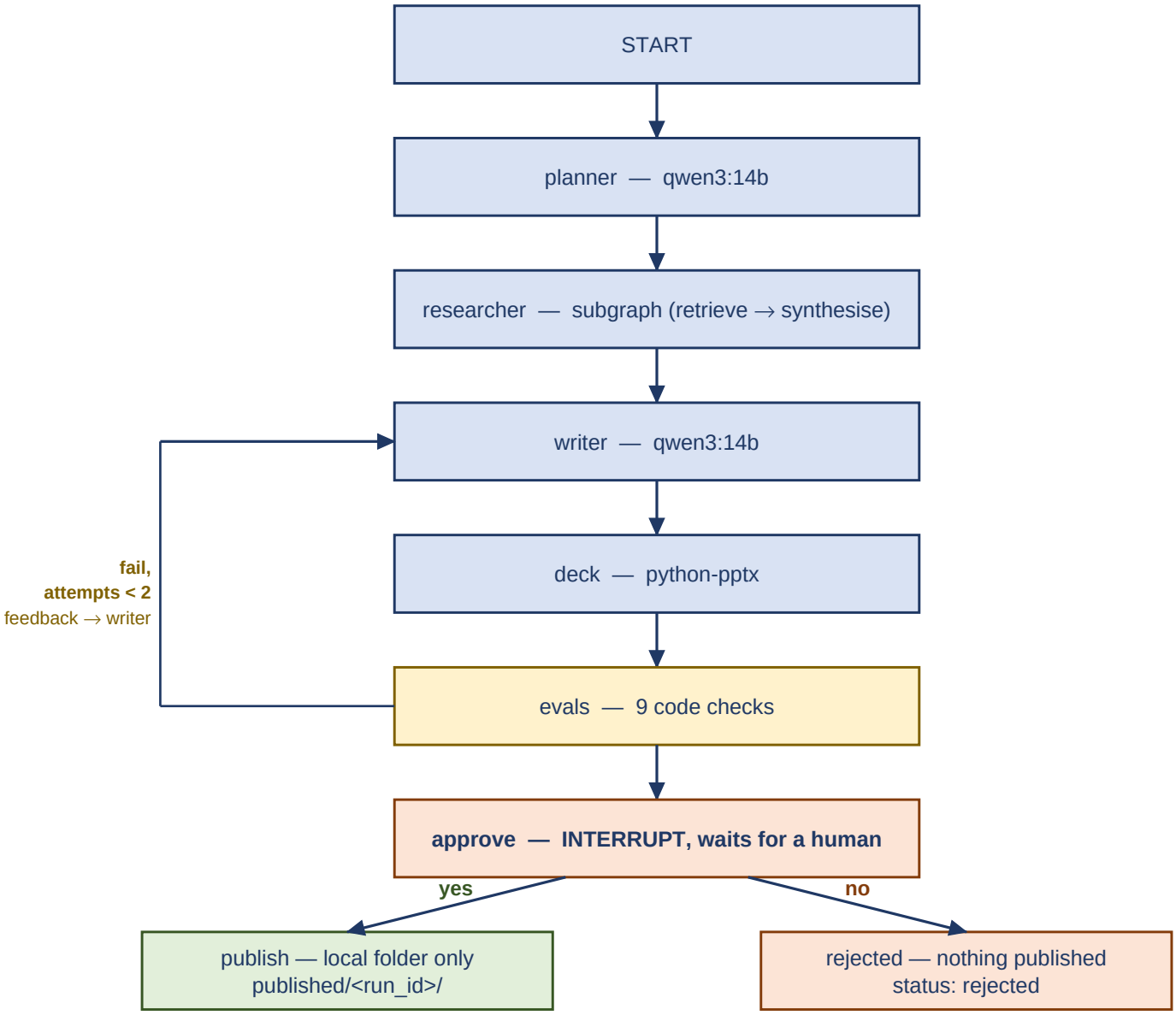
Do not memorise it. Walk the diagrams in order: the flow first (what the system does), then the researcher and the eval gate (how it stays honest), then the deployment (how it survives a crash), then the model table (why each model). If someone asks “draw the design”, draw diagram 1 from memory — the rest is detail you can add while talking.

The one-sentence version: a goal and an audience go in, a planner plans, a researcher retrieves the knowledge-base documents, a writer drafts with a citation on every factual sentence, code checks verify it, a deck is built, and a human approves or rejects — all as a FastAPI service in Docker, with a SQLite checkpoint so a crash resumes the run instead of restarting it.

## Inside


1 The whole graph — one flow, seven nodes
2 The researcher — a subgraph inside the graph
3 The eval gate — nine code checks, not a model judging itself
4 Deployment — the same graph, with a web front door
5 Crash-resume — the proof, step by step
6 Why these models — one choice per node

# 1 The whole graph — one flow, seven nodes




After 2 failed attempts the graph ENDS — status eval_failed, no deck, no publish.

*Figure 1 — The graph in marketing_pipeline/graph.py. State is MarketingState; the checkpoint is keyed by thread_id, which is also the run_id.*

*Say this: "The graph is the state machine. The retry loop is an edge, not a flag — evals fail, the edge routes back to the writer with feedback, max two attempts. The approval is an interrupt, not a check — the graph literally stops and waits for a human answer on the same run identity."*

## 2 The researcher — a subgraph inside the graph

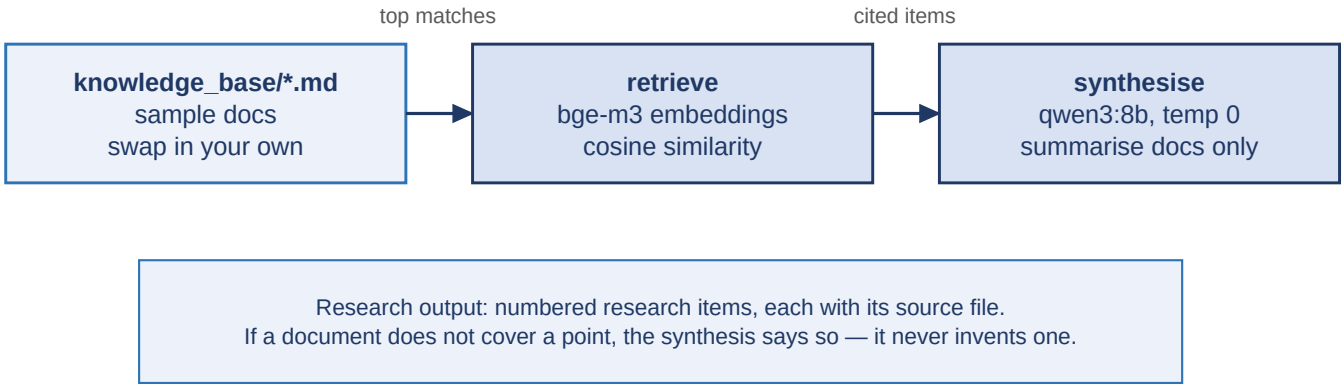



*Figure 2 — retrieve → synthesise, compiled as one node called "researcher" in the parent graph.*

*Say this: "Retrieval is the anti-hallucination layer. The writer never sees the open internet — only these research items. If the docs don't cover a point, the synthesis says so, and the writer has nothing to cite."*

## 3 The eval gate — nine code checks, not a model judging itself

| | | |
|---|---|---|
| 1 citations present | 2 citations valid | 3 all docs cited |
| 4 Sources section | 5 banned phrases = 0 | 6 PII patterns = 0 |
| 7 draft ≥ 200 chars | 8 citation coverage ≥ 0.5 | 9 deck slides distinct |

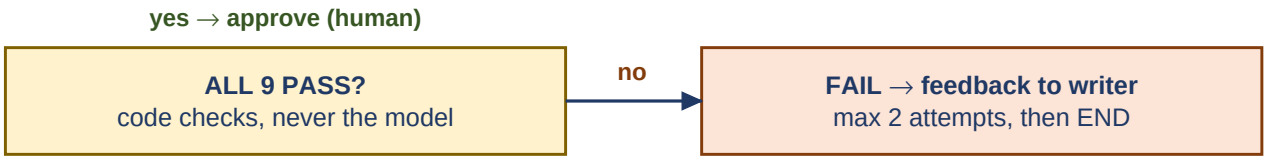



*Figure 3 — The gate sits after writer and deck, so it checks the deliverables, not just the draft.*

*Say this: "The gate is proven by failure, not just passes. Run-9a8b0e77 failed the deck-distinctness check twice and ended unpublished. And the ninth check was itself first wrong — it read slide titles too, so identical bodies passed. We test a check against a known-bad artifact before trusting it."*

# 4 Deployment — the same graph, with a web front door

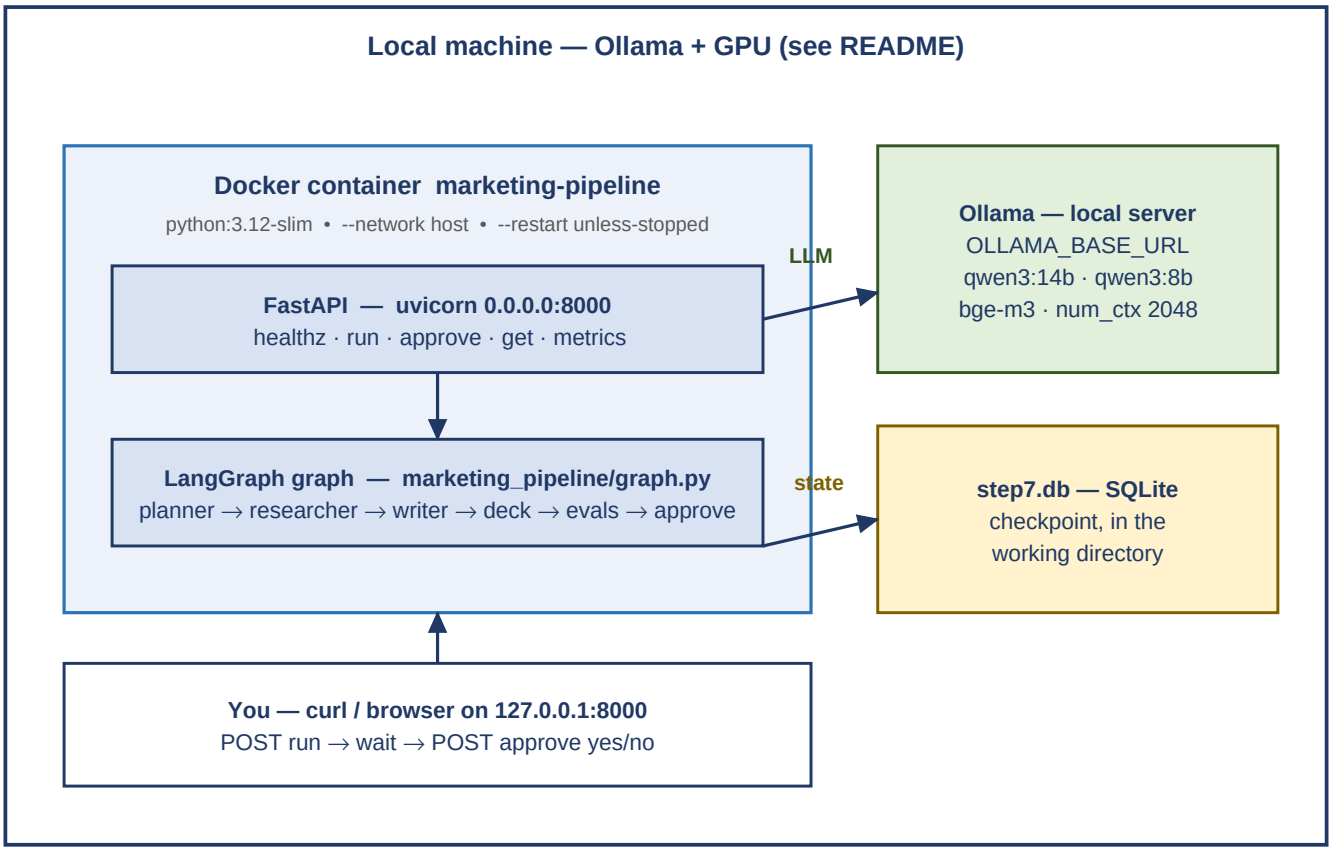



*Figure 4 — FastAPI is the front door, Docker is the box, SQLite is the memory, Ollama is the engine.*

Note: docker kill + docker start keeps the checkpoint — it lives on the host, not only inside the container. That is the whole point of the bind mount.

> *Say this: "Phase 2 is a graph you run from the command line. Phase 3 is the same graph with a web front door. FastAPI takes a request, starts a run, and remembers it by thread_id. Docker is just the box it runs in — one command, restarts itself if it dies."*

## The API

GET /healthz — service up, version, uptime, Ollama model list. POST /api/v1/marketing/run — start a run (goal, audience, services); returns run_id, status, deck path, next nodes, draft preview; pauses at approve. POST /api/v1/runs/{run_id}/approve — resume the same thread with yes/no (anything else: HTTP 400; an unmatchable service name: HTTP 422 with the reason). GET /api/v1/runs/{run_id} — current state of one run. GET /metrics — decks built and runs published (filesystem, survive restart) vs runs started in this process (memory, resets to 0).

# 5 Crash-resume — the proof, step by step

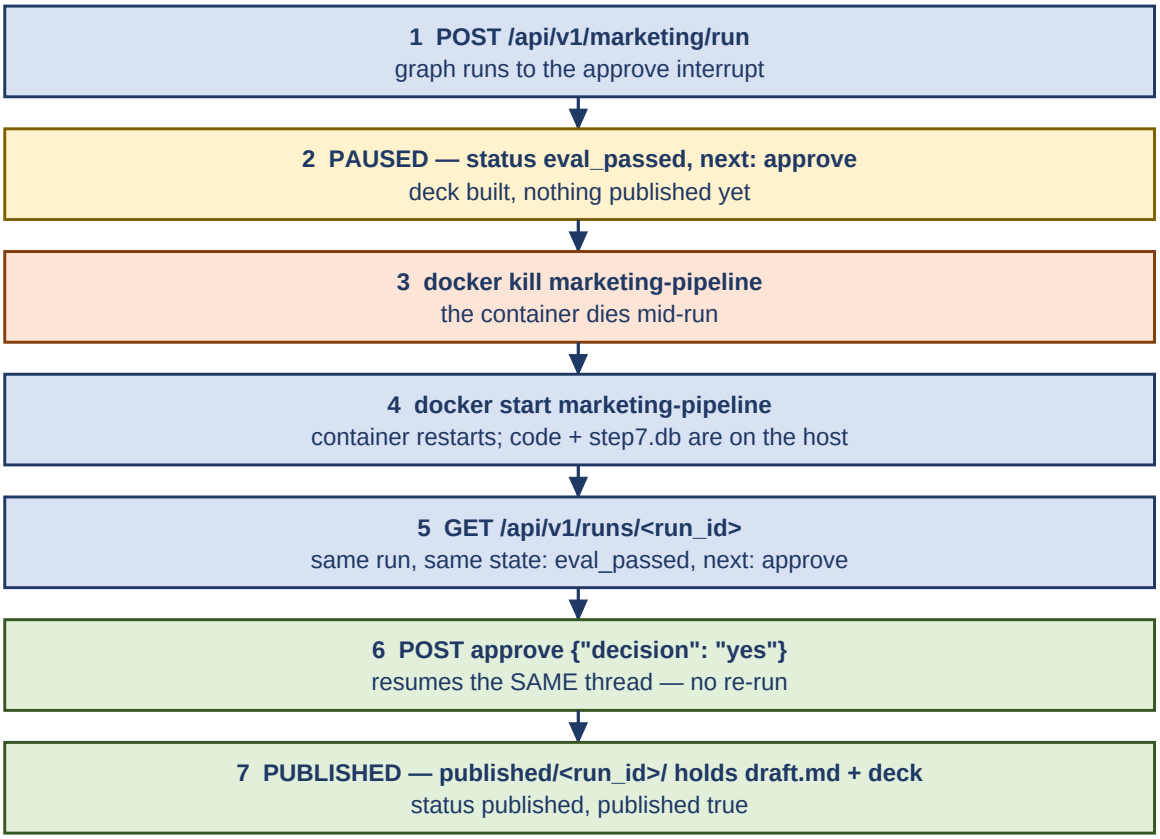



*Figure 5 — Proven 4 November 2026 with run-42f19498. The checkpoint lives in step7.db on the host, not only inside the container.*

*Say this: "The checkpoint is written after every step, on the host, bind-mounted into the container. Kill the container, start it again, and the same run is still there — deck ready, approval still pending. Approving after the restart publishes it. Same run identity, resumes where it stopped, never restarts from scratch."*

# 6 Why these models — one choice per node

| Node | Model | Job — why this one | Cost / run |
|---|---|---|---|
| **Planner** | **qwen3:14b** | Hardest job: turn a goal into a structured plan with only allowed services. A bad plan wastes every node after it — spend quality here. | ~25–35 s |
| **Retrieve** | **bge-m3** | Embeddings for the RAG subgraph. Proven embedder for this stack; cosine similarity in plain Python, no vector DB — the docs are short on purpose. | ~9 s |
| **Synthesise** | **qwen3:8b** | Summarising retrieved docs only. Easy job, small model is enough; temperature 0 so it stays faithful. | ~10 s |
| **Writer** | **qwen3:14b** | Drafting with strict citations. Hard enough to need 14b; the eval loop, not the first draft, guarantees quality. | ~70–80 s |
| **Deck** | **python-pptx (code)** | No model at all — the deck is built from the draft's own sections. Formatting is deterministic, not generated. | instant |
| **Evals** | **code, not a model** | 9 deterministic checks. Never the writer judging itself — a model judge can be talked into passing its own work. | instant |
| **Approve** | **a human** | The cheapest, most reliable judge for the final publish decision. Sits after evals, not instead of them. | — |

## The doctrine, in four lines

- **Cheapest model that passes.** Pick per node, never one model for the whole graph. If a node keeps failing eval, escalate that node only — never the whole graph.
- **Frontier for planning and judging, cheapest passing model for implementation.** A wrong plan or a wrong eval poisons everything downstream; a wrong draft gets caught by the eval loop and retried.
- **Local for this exercise, deliberately.** Three reasons: cost (0 SEK per run), privacy (the knowledge-base documents never leave the machine), and the point of the exercise — proving the deployment pattern works with the cheapest model that passes, so the pattern is not tied to any one provider.
- **The evals are the gate either way.** Local or frontier, the model choice is re-evaluated per task — and the evals decide whether it passes, not the model itself. Temperature 0 for anything that must be repeatable.

## What it costs

Nothing in money: local models on a local GPU box. Cost is tracked as tokens and seconds — a typical run is about 50–60 seconds of model time now (it was ~110 before the context fix made runs twice as fast). That 0 SEK is a property of this exercise, not a production cost model — the hardware and electricity are real, just already paid for.

## Measured results to quote

Held-out evals on five fresh topics the graph had never seen — manufacturing (coverage 0.68), public sector (0.90), energy (0.955), retail (0.758), Power BI / Power Apps (0.778–0.938). All passed all 9 checks on the first attempt, all human-approved and published locally. The honest limit, say it first: the evals check citations and safety, not whether the draft stuck to the brief — the manufacturing run passed while covering services outside its brief.

## What it cannot do — say this first

It is a learning exercise, not a production system. It cannot publish anywhere real. No authentication, no multi-user handling, no metrics database. It runs on one local machine that sleeps overnight. For

production: persistent metrics, API authentication, a brief-scope check, a queue, real endpoints.